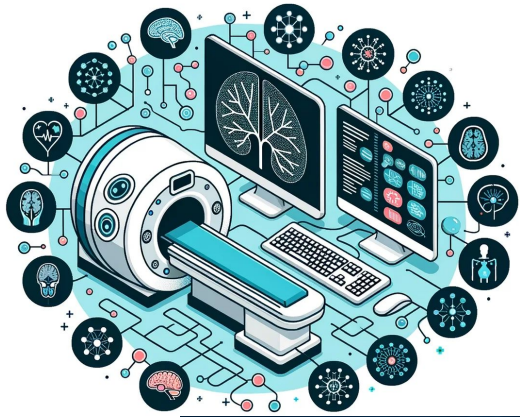


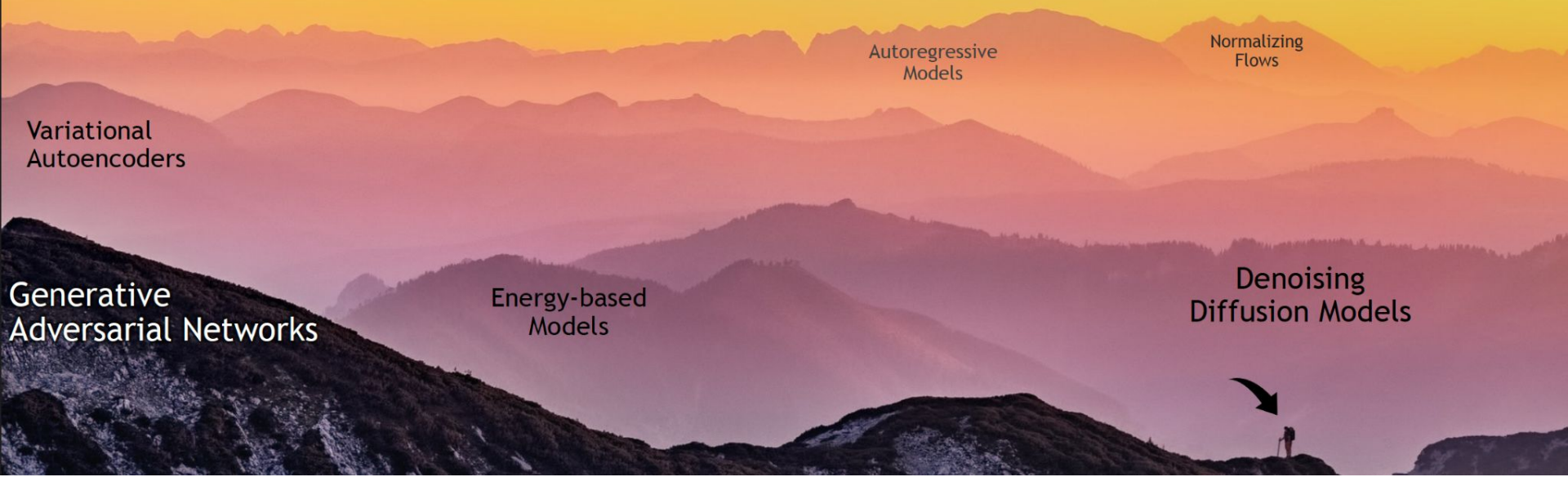


Frontiers in Generative AI for Medical Imaging and Healthcare

Session 4
Generative AI
Sep 27 2025



The Landscape of Deep Generative Learning



Variational
Autoencoders

Autoregressive
Models

Normalizing
Flows

Generative
Adversarial Networks

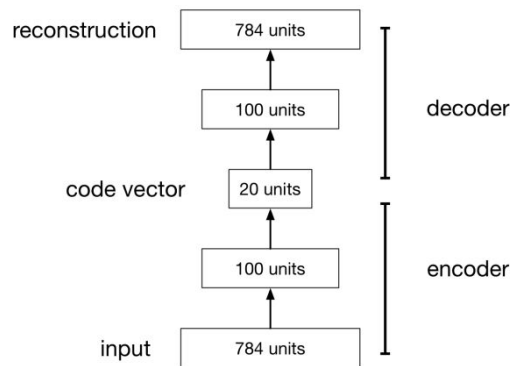
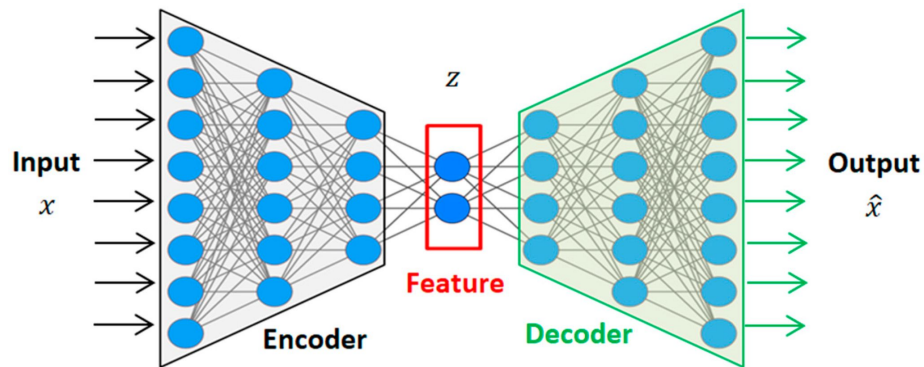
Energy-based
Models

Denoising
Diffusion Models



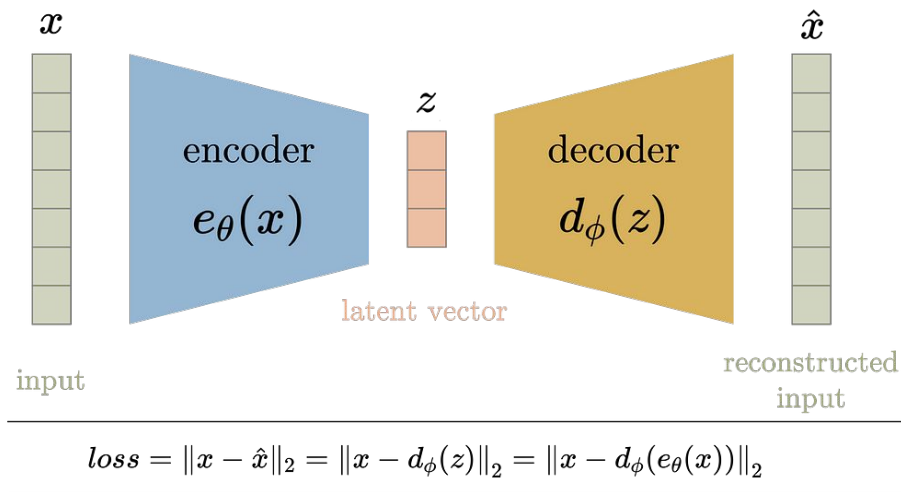
AutoEncoders

- AE is a feed-forward neural network that given input x learns to reconstruct x .
- There is a bottleneck layer whose dimensions are much smaller than the input dimensions.
- We are reducing 784 input dimensions to only 20 dimensions.
 - We are compressing, keeping only important features.
 - We are trying to reproduce x given using the extracted features.



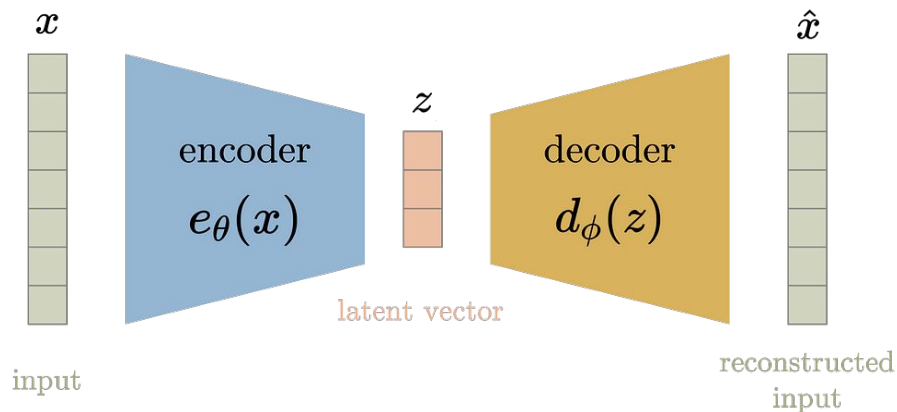
AutoEncoders

- Unsupervised neural network for learning compact embeddings of unlabeled data
- Two main components:
 - **Encoder $e_{\theta}(x)$** : compresses high-dimensional input x into a lower-dimensional latent vector z
 - **Decoder $d_{\phi}(z)$** : reconstructs x from z ensuring x retains most of the original information
- No requirement on the distribution of z it merely must allow accurate reconstruction



Encoder-Decoder Architecture

- Encoder
 - Sequence of user-defined layers (e.g. dense, convolutional, pooling)
 - Reduces input dimension from $\mathbb{R}^m$ to latent space $\mathbb{R}^n$ ($n < m$)
- Latent Space
 - Dimensionality n is a hyperparameter controlling compression level
 - Smaller $n \Rightarrow$ higher compression but risk of information loss
- Decoder
 - Near-complement of encoder layers (e.g. transposed conv for conv, unpooling for pooling)
 - Mirrors encoder in reverse to rebuild input dimensions

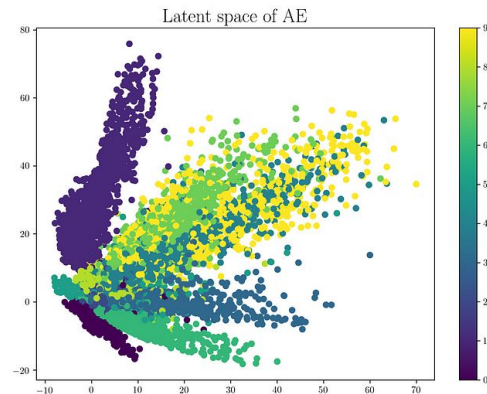
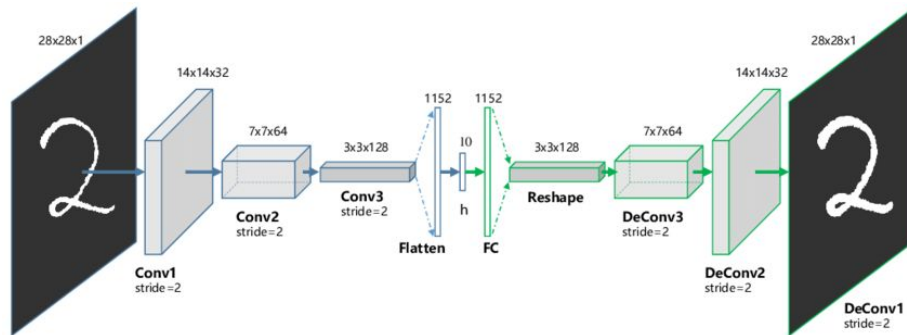


$$loss = \|x - \hat{x}\|_2 = \|x - d_{\phi}(z)\|_2 = \|x - d_{\phi}(e_{\theta}(x))\|_2$$

$$\mathcal{L}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|x_i - d_{\phi}(e_{\theta}(x_i))\|^2$$

Latent Space Visualization

- **Clustering:** Similar digits form distinct clusters in latent space
- **Unregularized Regions:** Empty areas (no training samples) yield meaningless outputs when decoded
- **Limited Generativity:** Only points within learned clusters produce valid digit variations; random samples elsewhere produce garbage
- **Decoding Reliability:** Once trained, there's no way to tell if a random latent vector will decode to a valid digit
- **Primary Use:** Autoencoders excel at compression (reducing redundancy) rather than generative modeling
- **Non-Linear Advantage:** Unlike PCA, non-linear AEs capture complex structures in lower dimensions with less information loss



Variational AutoEncoders

- **Traditional AutoEncoders**

- Deterministic encoding
- Point estimates in latent space
- No probabilistic interpretation
- Can't generate new samples easily

- **Variational AutoEncoders**

- Probabilistic encoding
- Distributions in latent space
- Principled probabilistic framework
- Natural generation mechanism

- **VAEs** learn a probability distribution over the latent space, not just point estimates. This allows us to sample new data points.



<https://xnought.github.io/vae-explainer/>

Variational AutoEncoders

- Encoder Network $q(Z|X)$
 - Maps input X to parameters of a distribution in a latent space
 - Outputs mean μ and standard deviation σ
 - Defines a Gaussian distribution $N(\mu, \sigma^2)$ in latent space
 - Latent Space Sampling
 - Sample Z from the distribution using the **reparameterization** trick
 - Decoder Network: $p(X|Z)$
 - Maps latent code Z back to data space to reconstruct X
 - VAEs model the joint probability distribution of data and latent variables
 - $p(X, Z) = p(X|Z) * p(Z)$
 - $p(Z)$ is the prior: Typically $N(0, 1)$
 - $q(Z|X)$ is the approximate posterior modeled by the encoder
 - $p(X|Z)$ is the likelihood modeled by the decoder
-

Reparameterization Trick

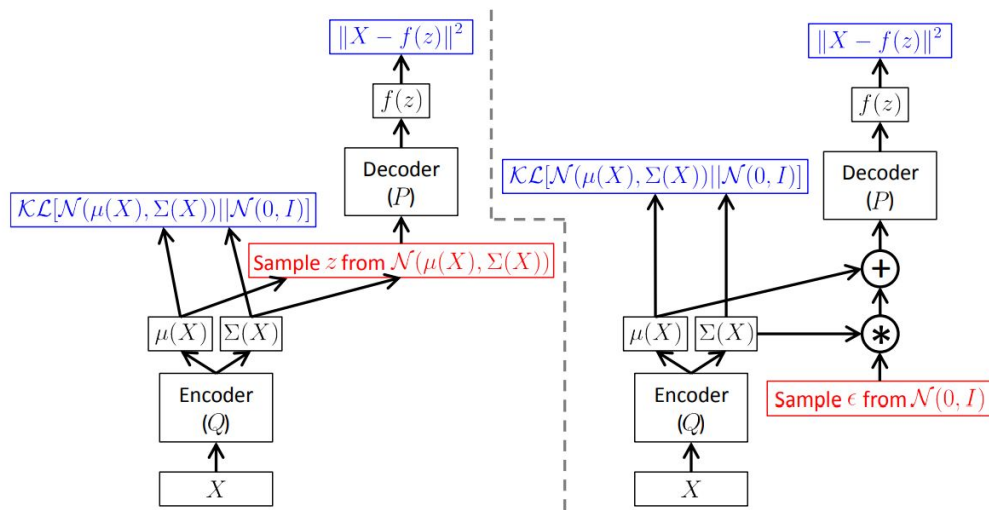
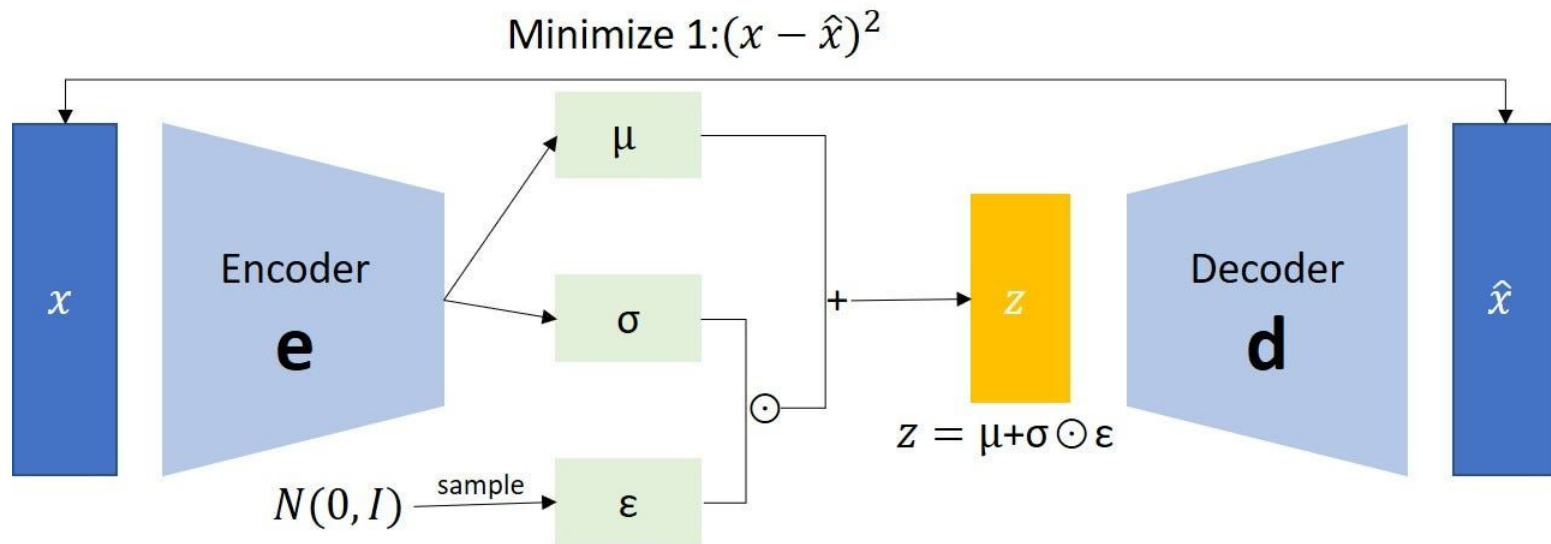


Figure 4: A training-time variational autoencoder implemented as a feed-forward neural network, where $P(X|z)$ is Gaussian. Left is without the “reparameterization trick”, and right is with it. Red shows sampling operations that are non-differentiable. Blue shows loss layers. The feedforward behavior of these networks is identical, but backpropagation can be applied only to the right network.

Variational AutoEncoders

- Reconstruction term
 - How can we reconstruct X from Z
 - Implemented as reconstruction loss (e.g., MSE)
 - Regularization term
 - How close is our encoding to the prior
 - Keeps latent space regular and smooth
 - $KL[q(Z|X) \parallel p(Z)]$
 - Training algorithm
 - Pass input X through encoder, get μ and σ
 - Sample ϵ from $N(0, 1)$ and compute $Z = \mu + \sigma * \epsilon$
 - Pass Z through decoder and compute losses (Reconstruction Loss + KL Divergence)
 - Backpropagate through decoder, backpropagate through reparameterization, and backpropagate through encoder to update the weights
-

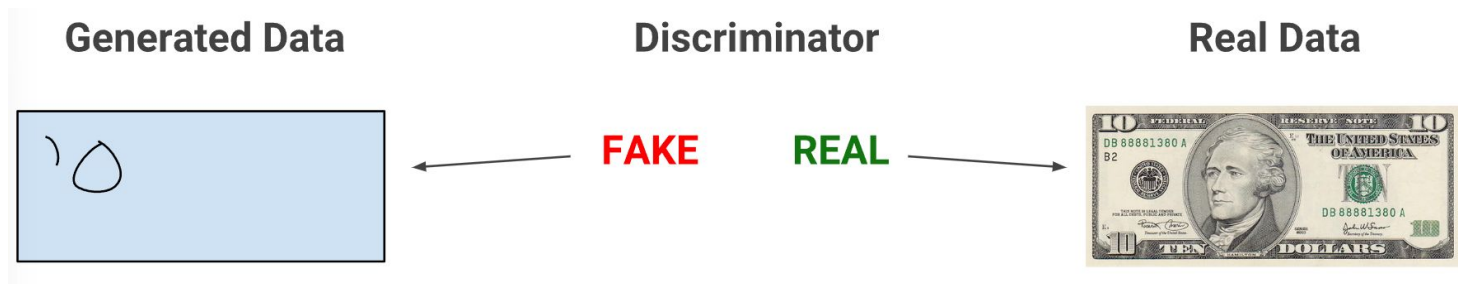
Variational AutoEncoders



Minimize 2: $\frac{1}{2} \sum_{i=1}^N (\exp(\sigma_i) - (1 + \sigma_i) + \mu_i^2)$

Generative Adversarial Network (GAN)

- GAN has two parts:
 - The **generator** learns to generate plausible data. The generated instances become negative training examples for the **discriminator**
 - The **discriminator** learns to distinguish the generator's fake data from real data. The **discriminator** penalizes the generator for producing implausible results.
- When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake!



Generative Adversarial Network (GAN)

- As training progresses, the generator gets closer to producing output that can fool the discriminator

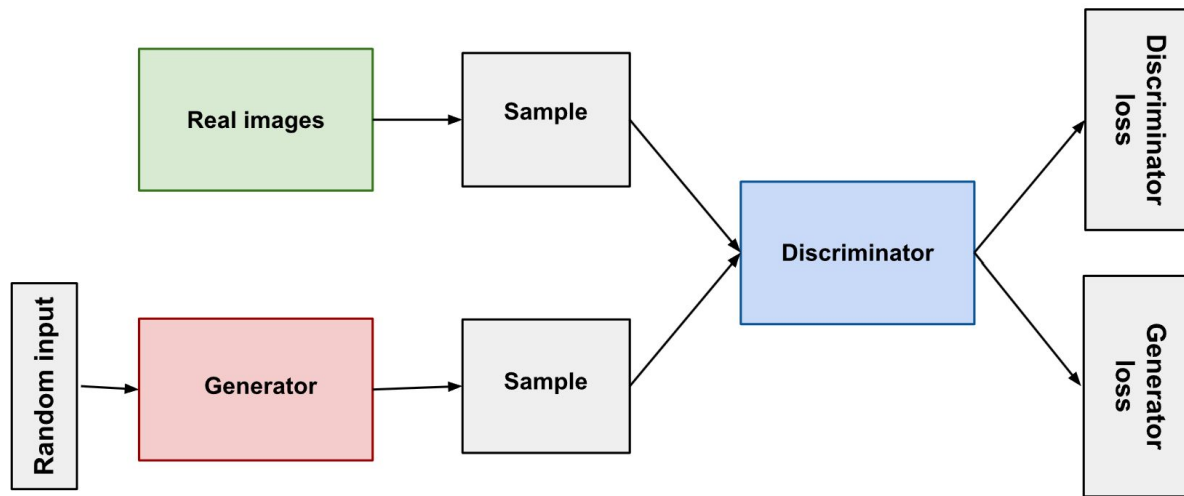


- If generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.



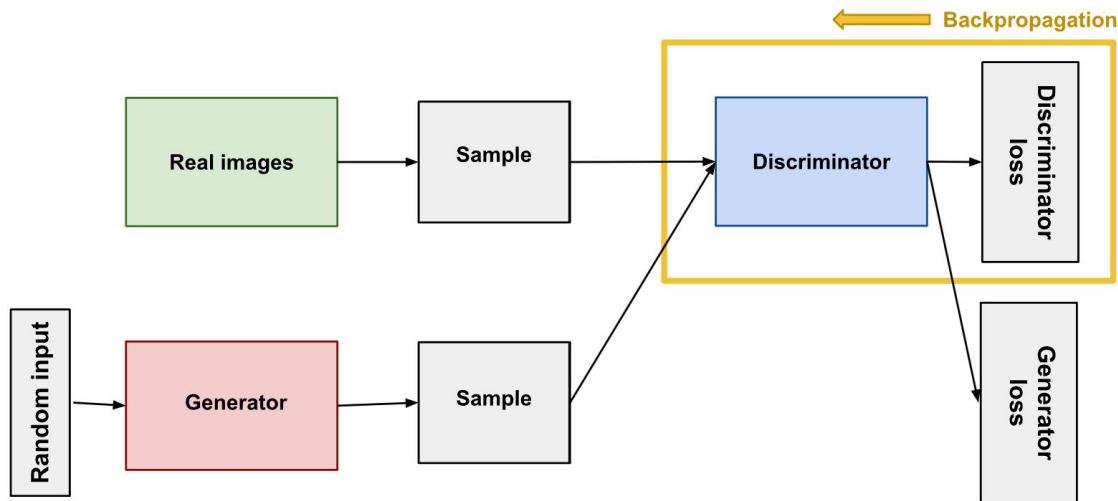
Generative Adversarial Network (GAN)

- Both **generator** and **discriminator** are neural networks.
- The **generator** output is connected directly to the **discriminator** input.
- The **discriminator's** classification provides a signal through **backpropagation** which **generator** uses to update its weights.



Generative Adversarial Network (GAN)

- The **discriminator** in a GAN is simply a classifier.
- It distinguished real data from data created by the **generator**.
- It could use any network architecture appropriate to the type of data it's classifying.



Generative Adversarial Network (GAN)

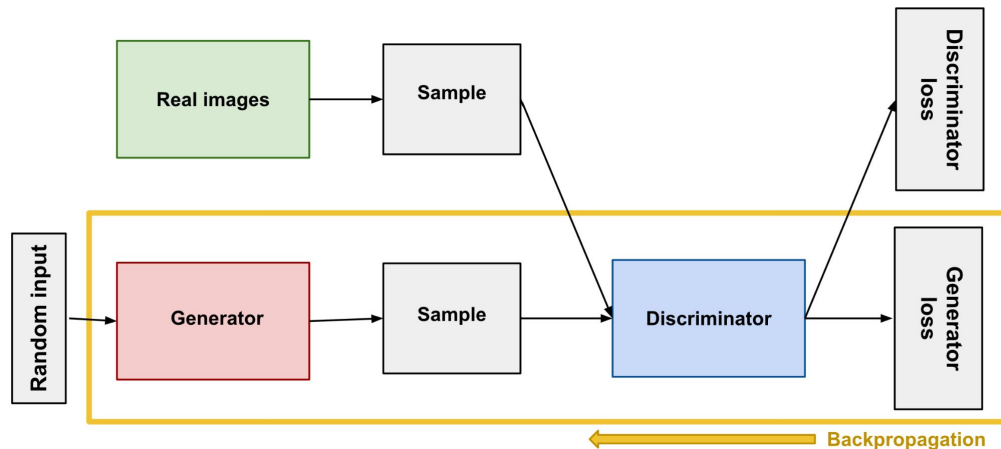
- The **discriminator** training data comes from two sources:
 - **Real data** instances, such as real pictures of people or real medical images. The **discriminator** uses these as positive examples during training.
 - **Fake data** instances created by the **generator**. The **discriminator** uses these as negative examples during training.
 - The discriminator connects to two loss functions:
 - The **discriminator** classifies both real data and fake data from the **generator**
 - The **discriminator** loss penalizes the **discriminator** for misclassifying a real instance as fake or a fake instance as real.
 - The **discriminator** updates its weights through **backpropagation** from the **discriminator** loss through the **discriminator** network.
 -
-

Generative Adversarial Network (GAN)

- The **generator** learns to create fake data that can fool the **discriminator**
 - Goal: Make the **discriminator** classify generator output as "real"
 - Random Input (Noise):
 - Usually sampled from simple distribution (e.g., uniform)
 - Lower dimensional than output space
 - Enables diverse data generation
 - **Generator Network**
 - Transforms random noise → meaningful data instances
 - By varying input noise → samples from different parts of target distribution
 - Output
 - Synthetic data matching the format of real data
 - Should be indistinguishable from real samples
-

Generative Adversarial Network (GAN)

- Training the **generator** - Adversarial Training
 - Sample random noise
 - Generate fake data from noise
 - Pass fake data through discriminator
 - Get discriminator's "Real/Fake" classification
 - Calculate generator loss (penalized when discriminator correctly identifies fakes)
 - Backpropagate through BOTH networks
 - Update ONLY generator weights



Adversarial Training Dynamics and Convergence

- Two players: **Generator (G)** vs **Discriminator (D)**
 - Alternating updates:
 - Train D (keep G fixed) → D learns G's flaws
 - Train G (keep D fixed) → G learns to fool a stationary D
 - Why alternate? Prevents G from chasing a moving target; turns a hard generation task into solvable classification steps.
 - **Bootstrapping**: if D can't beat random on G's early (bad) samples, training can't get started.
 - **Convergence behavior**: as G improves, D's accuracy → ~50% (can't tell real vs fake). After this point D's feedback becomes noise → risk of G quality collapse.
 - In practice, GAN "**convergence**" is often brief/unstable; monitor and stop before feedback degrades.
-

GAN Losses as Distances Between Distributions

- Goal: make P_g (generated) $\approx P_{data}$ (real).
 - G and D use related but different losses derived from one distance; G only affects the “fake” term.
 - Minimax (original GAN)
 - D maximizes: $\log D(x) + \log(1 - D(G(z)))$
 - G minimizes: $\log(1 - D(G(z)))$
 - Wasserstein GAN (WGAN)
 - Replace classifier with a critic (real-valued scores, not probabilities).
 - Critic maximizes: $D(x) - D(G(z))$
 - G maximizes: $D(G(z))$
 - **Benefits:** better gradients, fewer stalls/vanishing-gradient issues, more stable than minimax.
-

Common GAN Problems

- Vanishing gradients

- What happens: D gets too good $\rightarrow$ G gets almost no learning signal.
- Tries: Wasserstein loss (WGAN); modified minimax (maximize $\log D(G(z))$).

- Mode collapse

- What happens: G maps many z 's to the same few outputs; G/D chase each other and cycle.
- Tries: Wasserstein loss (train D to optimality without killing gradients); Unrolled GANs (anticipate future D steps).

- Failure to converge / instability

- What happens: oscillations or divergence instead of settling.
 - Tries: Regularization—add noise to D inputs; penalize D weights.
-

GANs in Medical Imaging

- **Clinical need:** CS-MRI speeds up scans by undersampling k-space, but classic sparsity methods struggle with detail, tuning, and speed. **DAGAN** brings a conditional GAN to de-aliasing for fast, high-quality recon.
 - **Key contributions:** U-Net generator with skips; **refinement learning** for stability/speed; content loss (pixel, frequency, VGG) + adversarial; frequency-domain consistency; broad comparisons.
 - **Speed:** ~0.2 s/CPU slice or ~5 ms/GPU slice → near real-time.
-

TABLE I
QUANTITATIVE RESULTS (NMSE AND PSNR) OF THE COMPARISON STUDY USING
DIFFERENT RANDOM UNDERSAMPLING RATIOS OF THE 1D GAUSSIAN MASK

Method	10%		20%		30%		40%		50%		Testing Time (30%)
	NMSE	PSNR	NMSE	PSNR	NMSE	PSNR	NMSE	PSNR	NMSE	PSNR	CPU (sec) / GPU (ms)
ZF	0.37±0.09	28.22±4.30	0.21±0.06	34.04±4.58	0.16±0.04	35.61±4.79	0.14±0.04	37.65±4.92	0.10±0.03	40.65±4.95	0.002±0.003 / —
TV	0.37±0.09	28.23±4.30	0.20±0.06	34.09±4.57	0.16±0.04	35.68±4.79	0.13±0.04	37.81±4.92	0.09±0.03	40.90±4.95	10.5±1.2 / —
SIDWT	0.36±0.10	28.96±4.79	0.15±0.04	36.99±5.34	0.11±0.03	38.93±5.45	0.10±0.03	40.51±6.31	0.07±0.02	43.53±6.26	126.0±22.7 / —
RecPF	0.36±0.08	28.62±4.46	0.13±0.02	36.21±5.57	0.11±0.02	37.88±4.90	0.09±0.03	41.37±5.88	0.06±0.02	44.78±6.12	35.9±0.8 / —
DLMRI	0.39±0.10	28.25±4.24	0.16±0.03	36.54±4.21	0.12±0.02	38.24±4.29	0.09±0.02	42.11±4.17	0.06±0.01	45.40±4.08	49.1±9.8 / —
PBDW	0.21±0.05	31.14±3.65	0.11±0.04	37.56±3.07	0.10±0.04	38.07±3.02	0.07±0.02	41.91±2.95	0.04±0.01	46.05±3.05	223.5±45.5 / —
PANO	0.22±0.05	30.84±4.48	0.09±0.02	38.54±4.04	0.08±0.02	39.16±4.02	0.05±0.01	43.05±3.64	0.04±0.01	45.44±3.42	108.6±17.4 / —
Noiselet	0.25±0.07	30.19±5.40	0.13±0.04	36.97±5.94	0.10±0.03	39.01±5.96	0.08±0.03	41.51±6.84	0.06±0.02	44.44±6.79	31.1±0.5 / —
BM3D	0.20±0.05	31.72±5.23	0.10±0.02	39.40±4.58	0.09±0.02	39.93±4.59	0.07±0.01	43.66±4.48	0.05±0.01	45.89±4.41	87.5±3.0 / —
DeepADMM	0.24±0.03	30.70±4.78	0.16±0.09	37.31±3.07	0.15±0.04	37.36±3.27	0.10±0.06	42.36±2.41	0.09±0.04	42.60±2.60	3.2±0.2 / —
PG	0.19±0.04	32.82±3.80	0.09±0.02	39.10±3.84	0.09±0.02	39.72±4.30	0.06±0.01	43.28±4.17	0.05±0.01	44.39±4.18	0.2±0.1 / 5.4±0.1
PPG	0.23±0.06	31.33±2.81	0.10±0.03	38.88±3.41	0.09±0.02	39.78±3.95	0.07±0.03	42.31±2.98	0.05±0.01	45.61±3.73	0.2±0.1 / 5.4±0.1
PPGR	0.19±0.04	32.84±4.46	0.10±0.02	39.05±3.82	0.09±0.02	39.53±4.12	0.05±0.01	44.45±4.14	0.04±0.01	47.30±3.93	0.2±0.1 / 5.4±0.1
PFPGR	0.17±0.03	33.79±4.26	0.09±0.02	39.44±3.95	0.08±0.02	40.20±4.07	0.05±0.01	44.83±4.16	0.04±0.01	47.83±4.10	0.2±0.1 / 5.4±0.1

CS-MRI as an Inverse Problem

- Forward model $Fx + \varepsilon = y,$

- Classic formulation

$$\min_x \frac{1}{2} \|F_u x - y\|_2^2 + \lambda \mathcal{R}(x),$$

- Deep learning-aided CS-MRI: add a CNN prior/regularizer.

$$\min_x \frac{1}{2} \|F_u x - y\|_2^2 + \lambda \mathcal{R}(x) + \zeta \|x - f_{\text{cnn}}(x_u | \hat{\theta})\|_2^2,$$

From GANs to Conditional GANs for MRI

- GAN objective (minimax):

$$\min_{\theta_G} \max_{\theta_D} \mathcal{L}(\theta_D, \theta_G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D_{\theta_D}(\mathbf{x})] \\ + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D_{\theta_D}(G_{\theta_G}(\mathbf{z})))]$$

- Gradient trick

$$\Delta \theta_G = \nabla_{\theta_G} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [-\log D_{\theta_D}(G_{\theta_G}(\mathbf{z}))].$$

- JSD link

$$\min_{\theta_G} \mathcal{L}(\theta_D^*, \theta_G) = \text{JSD}(p_{\text{data}} || p_G) - \log(2)$$

- Conditional GAN loss for DAGAN: condition the generator on zero-filled aliased input

$$\min_{\theta_G} \max_{\theta_D} \mathcal{L}_{\text{cGAN}}(\theta_D, \theta_G) = \mathbb{E}_{\mathbf{x}_t \sim p_{\text{train}}(\mathbf{x}_t)} [\log D_{\theta_D}(\mathbf{x}_t)] \\ + \mathbb{E}_{\mathbf{x}_u \sim p_G(\mathbf{x}_u)} [-\log(D_{\theta_D}(G_{\theta_G}(\mathbf{x}_u)))],$$

DAGAN Architecture

- U-Net generator (8 enc/8 dec layers, skips, BN, LeakyReLU, tanh out);
- CNN discriminator (11 conv + BN/LeakyReLU, sigmoid).

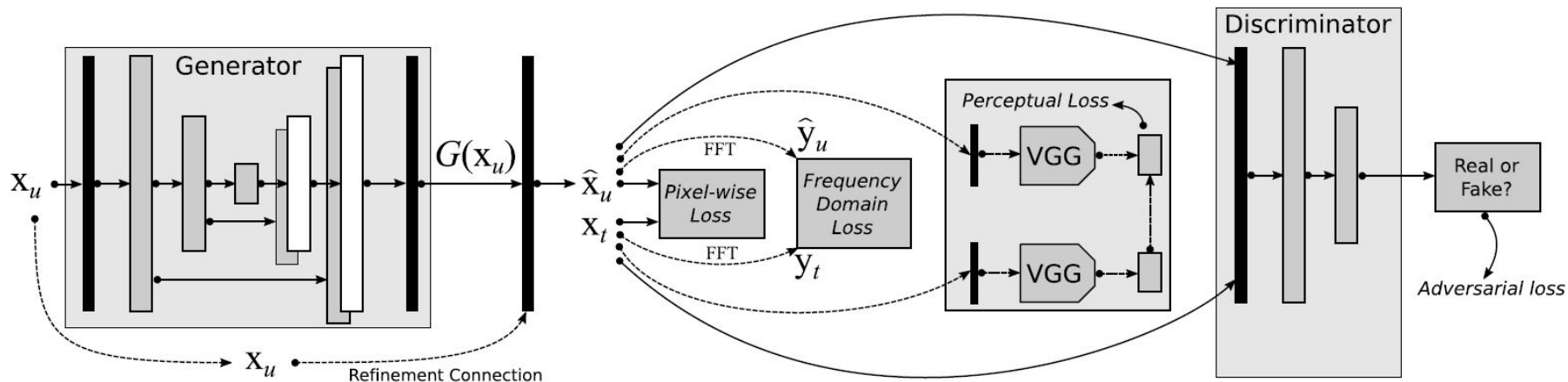


Fig. 1. Schema for our proposed conditional GAN-based de-aliasing for fast CS-MRI (DAGAN).